

Unit 2: - Fundamentals of Angular

What is Angular?

Angular is a **TypeScript-based front-end framework** developed by **Google**.

It helps us build **Single Page Applications (SPAs)** — where the page doesn't fully reload when navigating between different views.

Tools to Install

- **Node.js** – JavaScript runtime environment
- **npm** – Node package manager (comes with Node.js)
- **Angular CLI** – Command Line Interface to create/manage Angular apps

Installation Commands

```
npm install -g @angular/cli
```

Create Your First Angular App

```
ng new my-first-app  
cd my-first-app  
ng serve
```

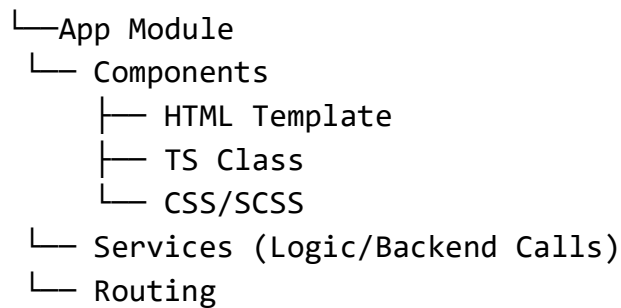
- To open in the browser automatically: `ng serve -o`
- To run multiple Angular projects at the same time:
`ng serve --port 4201` **URL to see your app:** <http://localhost:4200>

Architectural Overview:-

- **Component-based architecture** – UI is divided into reusable building blocks.
- **Modules** – Organize components and features.
- **Component Structure:**
`Template (HTML) + Class (TS) + Metadata = Component`
- **Dependency Injection (DI)** – Services are provided automatically to components.
- **Data Binding** – Sync between HTML and TypeScript.
- **SPA vs MPA:** Angular builds **SPAs**. (Single Page Application)

Diagram

Src



Important Files in Angular Project

File	Purpose
package.json	Lists packages. dependencies → part of production build, devDependencies → only for local development.
angular.json	Project configuration (styles, scripts, build settings, entry point).
styles.css	Global CSS styles (affects all components).
.gitignore	Files/folders to exclude from version control.
tsconfig.json	TypeScript configuration (compiler options).
.editorconfig	Editor/VS Code settings.
index.html	Main HTML file that loads Angular (<app-root> placeholder).
main.ts	Entry point of the app. Bootsrap the root module.
app.config.ts	App configuration (environment, providers).

Components:- A **component** is a UI block made up of:

- **HTML** – View
- **TS** – Logic
- **CSS** – Styling
- **Decorator** – Metadata (@Component)

Create Component:- ng generate component student-list

Generated Files:

- student-list.component.ts – Logic
- student-list.component.html – UI
- student-list.component.css – Styles

- student-list.component.spec.ts – Unit test

Use in App

```
<!-- app.component.html -->
<app-student-list></app-student-list>
```

Data Binding

- Data Binding connects **UI** and **Code**. Imagine you have a **TV remote** and a **TV** . When you press a button on the remote, the TV changes.
- That's exactly how **data binding** works in Angular – it connects your **TypeScript code (remote)** with your **HTML view (TV screen)**.

Types of Data Binding:- One Way & Two Way

Type	Syntax	Purpose
Interpolation	{{ value }}	Show data from TS in HTML
Property	[property]="value"	Bind HTML property to TS variable
Event	(event)="handler()"	Call function on event
Two-way	[(ngModel)]="value"	Sync data both ways

- **Interpolation ({{ }})** :- Use curly brackets to show data from your code into HTML.

Suppose in admin component: -

```
<!-- admin.ts-->
```

```
export class AdminComponent {
  myName: string = "Jay";
}
```

```
<!-- admin.html -->
```

```
<p>Hello, my name is {{ myName }}</p>
```

Output: Hello, my name is Jay

- **Property Binding ([])** :- Like giving energy from code to HTML property.

```
<!-- admin.ts-->
export class AdminComponent {
  imageUrl: string = "https://picsum.photos/200";
}
```

```
<!-- admin.component.html -->
<img [src]="imageUrl">
```

Output:- An image appears from that link.

- **Event Binding (()):-** Like listening when someone presses a button and doing something.

```
<!-- admin.ts-->
export class AdminComponent {
  sayHello() {
    alert("Hello!");
  }
}
```

```
<!-- admin.html-->
```

```
<button (click)="sayHello()">Click Me</button>
```

Output: When you click, an alert pops: **Hello !**

- **Two-Way Binding [()] :-** Like a walkie-talkie– you can talk and listen. You need to import in components FormsModule

```
<!-- admin.ts-->
export class AdminComponent {
  myText: string = "";
}
```

```
<!-- admin.html-->
```

```
<input [(ngModel)]="myText" placeholder="Type something">
<p>You typed: {{ myText }}</p>
```

Output:

Whatever you type in the input instantly shows below.

- **Signals in Angular:** - Signals are like a **magic notebook** .
when you change something inside it, **everyone reading it sees the new value automatically**.

<!-- admin.ts-->

```
import { signal } from '@angular/core';
```

```
export class AdminComponent {
  firstName = signal("Signals Used");
```

```
  changeName() {
    this.firstName.set("Jay");
  }
}
```

<!-- admin.html->

```
<p>Hello {{ firstName() }}</p>
```

```
<button (click)="changeName()">Change Name</button>
```

Output:

- First shows: **Signals Used**
- After click: **Hello Jay**

Modules:-

- **Before Angular 15** → Needed **NgModules** (app.module.ts)
- **Angular 15+ / Angular 20** → Modules are **optional**.
- You can now use **Standalone Components**.

In Short:

- Old Angular → Needed a **school bag (module)** to carry books (components).
- New Angular → Each **book (component)** can be carried on its own.

Where are Modules in Angular 20?

- They still **exist** (for backward compatibility).
- But for new projects, **standalone components replace modules**.
- Modules are now **optional**.

Simple way to remember:

- **Old Angular (before v15)** → You needed a **school bag (module)** to carry your books (components).
- **New Angular (15+)** → Every book (component) is strong enough to be carried without a bag (standalone).

Directives: - Think of directives like magic wands that can **change your web page** without rewriting it.

Structural Directives: - Structural = change the structure of your page (add, remove, or repeat things)

***ngIf** → Show or hide something

- Imagine you have a **toy box**.
- If *ngIf is true → you **show the toy**
- If *ngIf is false → you **hide the toy**

```
<p *ngIf="isHappy">I am happy!</p>
```

- If isHappy = true → “I am happy!” shows
- If isHappy = false → it disappears

***ngFor** → Repeat something for many items

- Imagine you have **5 candies**, and you want to show each one.
- Instead of writing 5 <p> tags, *ngFor will **loop automatically**

```
<li *ngFor="let candy of candies">{{ candy }}</li>
```

- If candies = ['Choco', 'Lollipop', 'Toffee'] → it will make 3 list items

***ngSwitch** → Choose one thing from many (like a game of “Pick One”)

- Imagine you have 3 doors. Depending on which number you choose, you go through **that door only**

```
<div [ngSwitch]="day">
  <p *ngSwitchCase="Mon">Monday fun!</p>
  <p *ngSwitchCase="Tue">Tuesday fun!</p>
  <p *ngSwitchDefault>Weekend!</p>
</div>
```

- If day = 'Mon' → only Monday message shows

Attribute Directives: - Attribute = change the look or style of something ngClass → Add or remove CSS classes dynamically

- Imagine you have a **color-changing T-shirt**.
- You can make it **red, blue, or green** whenever you want

```
<p [ngClass]="{'red-text': isRed}">Hello!</p>
```

- If isRed = true → text becomes red
- If isRed = false → text goes back to normal

ngStyle → Change styles directly

- Like painting a wall **any color you want instantly**

```
<p [ngStyle]="{'color': textColor, 'font-size.px': fontSize}">Hello!</p>
```

- If textColor = 'blue' and fontSize = 20 → text becomes **blue and 20px**

In Short:-

- **Structural** → Changes **what is on the page**
- **Attribute** → Changes **how it looks**

Routing: - negation between one page to another.

Imagine your house has many **rooms**:

- Living Room
- Kitchen
- Bedroom

You don't need to build **a new house** for each room...You just **walk through the door** to go to the right room.

In Angular, **routing is like doors** that take you to different **pages (components)** inside one single app.

Crate one component like Admin, about

Then in **app.routes.ts** file:-

```
import { Routes } from '@angular/router';

import { AdminComponent } from './admin/admin.component';

import { AboutComponent } from './about/about.component';

export const routes: Routes = [

  { path: 'admin', component: AdminComponent },    // Admin page

  { path: 'about', component: AboutComponent } // About page

];
```

Then you need to import RouterOutlet and RouterLink in app.ts file

```
import { Component } from '@angular/core';

import { RouterOutlet } from '@angular/router';

import { RouterLink } from '@angular/router';
```



```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink], // ✅ Needed for routing
  template: `
    export class AppComponent {}
```

In App.html file

```
<ul>
  <li>
    <a routerLink="/admin">Admin</a>
    <a routerLink="/about">Admin</a>

  </li>
</ul>
<router-outlet/>
```